

A.A.P.T.: Advanced Attention-based Paging Technique for Optimized Memory Management in Dynamic Workloads

Pragun Kathuria, Avani Gupta, Ashank Singh, Tanvi Pandey

Department of Computer Science and Engineering

Shiv Nadar Institution of Eminence

CSD204 - Operating Systems

Email: pk944@snu.edu.in, ag995@snu.edu.in, as315@snu.edu.in, tp561@snu.edu.in

Abstract—This report introduces A.A.P.T. (Advanced Attention-based Paging Technique), a memory management simulation inspired by modern LLM infrastructure research. Traditional contiguous allocation suffers from significant external fragmentation, severely limiting throughput in high-concurrency environments. AAPT utilizes a non-contiguous, paged architecture to bridge classic OS paging concepts with the dynamic memory demands of modern AI-driven workloads. This project will implement a C-based simulator to evaluate the efficiency of paged allocations against traditional baseline heuristics.

Index Terms—Memory Management, Paging, External Fragmentation, Large Language Models, Operating Systems

I. INTRODUCTION

Memory management remains a critical bottleneck in operating system performance, directly dictating system throughput and the kernel’s ability to handle concurrent processes. Traditionally, operating systems allocate dynamic memory in contiguous blocks to simplify addressing logic and hardware-level translation. However, as computational workloads evolve—particularly with the deployment of highly concurrent pipelines and Large Language Models (LLMs)—the limitations of contiguous allocation have become glaringly apparent.

Modern applications are characterized by their “bursty” and unpredictable data buffers. When memory is strictly contiguous, the unpredictable lifecycle of these processes leads to severe external fragmentation. Over time, the physical memory space becomes riddled with small, unusable holes. This project investigates the necessary transition from these classical allocation models toward a non-contiguous, paged architecture inspired by recent breakthroughs in AI memory management, specifically PagedAttention. By building a functional simulator in C, this project aims to quantify the exact performance gains achieved by abandoning contiguous constraints.

II. LITERATURE SURVEY AND RELATED WORK

A. Traditional Allocation Heuristics and Limitations

Operating systems historically rely on foundational heuristics such as First-Fit, Best-Fit, and Worst-Fit for dynamic memory allocation.

- **First-Fit** allocates the first available block that satisfies the request, which is computationally fast but rapidly clutters the lower address spaces with small fragments.
- **Best-Fit** searches for the smallest adequate block, minimizing immediate waste but generating microscopic “sliver” holes that are entirely unusable by future processes.

While simple to implement, these methods inevitably succumb to severe external fragmentation. As comprehensively documented in Wilson et al.’s critical review of dynamic storage allocation [3], contiguous allocation strategies eventually fail under high-load conditions because the physical memory space becomes riddled with microscopic, unusable holes. In such scenarios, the system is forced to deny memory requests—crashing or stalling processes—even when the aggregate total of free memory across the system is more than sufficient.

B. Evolution of PagedAttention Architecture

Recent advancements in AI infrastructure have revolutionized how we handle dynamic memory. **PagedAttention (Kwon et al., 2023)** identified that memory waste in LLM serving is primarily due to the Key-Value (KV) cache [1]. Traditional frameworks allocated the KV cache contiguously based on the maximum possible sequence length. Because generated responses are highly variable, this resulted in massive internal fragmentation (memory reserved but never written to) and external fragmentation.

PagedAttention solved this by partitioning the memory pool into small, fixed-size physical blocks. By managing these blocks via a logical-to-physical mapping system—functionally identical to an OS Page Table—they decoupled logical contiguity from physical contiguity. This architectural shift demonstrated a near 40% increase in memory utilization, allowing systems to handle significantly larger batches of requests simultaneously.

C. Advancements via vAttention

Building upon the block-mapping concept, **vAttention (Prabhu et al., 2024)** proposed integrating these paged concepts directly into the OS’s existing virtual memory subsystem

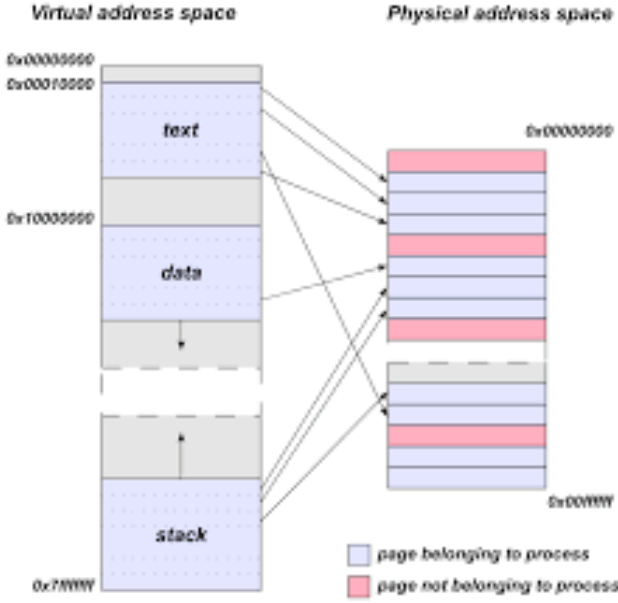


Fig. 1. Proposed OS Page Table Mapping: Translating contiguous logical requests to non-contiguous physical RAM frames.

[2]. Instead of managing custom memory pools at the application layer, vAttention leverages standard demand-paging. It allows memory buffers to grow dynamically by committing physical pages only when a process actively writes to them. This eliminates the need to pre-allocate massive chunks of RAM, showcasing how deep kernel-level OS features can be modernized for bursty data generation.

D. Current Progress in the Field

The progress achieved in this area demonstrates a clear paradigm shift toward "Application-Aware Paging." By treating isolated application buffers as virtual memory pages, systems can achieve near-zero external fragmentation. This project builds upon these specific research milestones by implementing a robust simulation in C, allowing us to isolate and quantify the benefits of such paged systems against standard contiguous algorithms.

III. PROPOSED METHODOLOGY: THE AAPT FRAMEWORK

To practically demonstrate the advantages of paged architectures, the AAPT framework will be developed as a multi-module C simulator.

- **Physical Memory Simulator:** A static array-based representation of a bounded RAM pool. This pool will be logically partitioned into discrete, fixed-size page frames (e.g., 32-byte or 64-byte blocks).
- **The Mapping Engine:** A central Page Table structure responsible for tracking the state of physical frames. It will map a process's logical request IDs to an array of non-contiguous physical block indices.
- **Baseline Comparison Module:** A parallel implementation of a First-Fit contiguous allocator. This will serve as

the control group, processing the exact same workload as the paged engine to provide direct comparative metrics.

IV. EXPERIMENTAL DESIGN AND METRICS

In alignment with the project requirements to prove algorithmic superiority, AAPT will be evaluated against the baseline First-Fit allocator using a simulated bursty workload. The experiments will measure:

- 1) **System Throughput:** The total number of memory requests successfully processed and completed per second under high concurrency.
- 2) **Memory Utilization Ratio:** The percentage of total physical memory successfully occupied by active data versus the percentage lost to fragmentation.
- 3) **Allocation Latency:** The computational time complexity ($O(n)$ vs $O(1)$) required to search for and map free memory blocks.

V. CONCLUSION

By bridging the gap between classic OS paging theory and cutting-edge AI memory management, AAPT seeks to demonstrate that non-contiguous allocation is essential for maximizing throughput in the next generation of systems. The experimental phase is expected to showcase a drastic reduction in allocation failures, proving that paged architectures provide a highly efficient, scalable alternative to traditional contiguous memory management.

REFERENCES

- [1] Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," *SOSP*, 2023.
- [2] Prabhu et al., "vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention," 2024.
- [3] P. R. Wilson, M. S. Johnstone, M. Neely, and D. Boles, "Dynamic Storage Allocation: A Survey and Critical Review," in *International Workshop on Memory Management*, 1995.
- [4] Silberschatz, A., Galvin, P. B., and Gagne, G. *Operating System Concepts*, 10th Edition, Wiley.